DATA ANALYSIS PORTFOLIO PROJECT

Cleaning and Analysing 10,000 Café Transactions

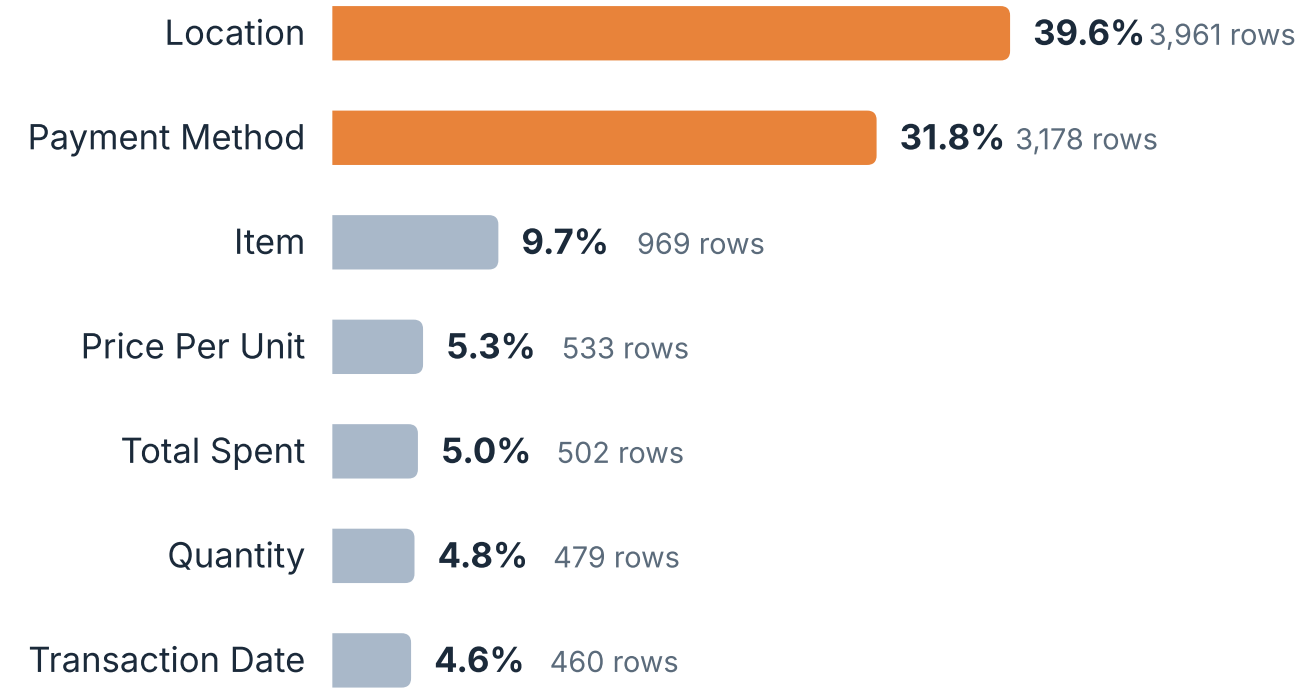
From a deliberately messy dataset to a trustworthy dashboard — using SQL and Excel

Dataset: Kaggle, “Cafe Sales – Dirty Data for Cleaning Training” · Tools: SQLite (DB Browser) and Microsoft Excel

THE PROBLEM

Every column was dirty — and two of them were mostly beyond repair

Share of values that were ERROR, UNKNOWN or blank, by column (10,000 rows)



10,000

transactions, 8 columns, imported as plain text

7,139

bad Payment Method and Location values that no other column can rebuild

Bad values were hidden in three disguises: the words **ERROR** and **UNKNOWN**, and empty cells. Orange = cannot be recovered from the other columns.

THE METHOD

Recover first, label second, delete never

1 Protected the raw data

Copied the table before touching anything. The raw table stays as the backup.

2 Recovered items from prices

Cookie, Tea, Coffee and Salad each have a unique price. **468** missing items identified.

3 Recovered prices from items

Every item has one fixed price. **479** missing prices filled in.

4 Rebuilt the maths

Total = Quantity × Price. **479** totals and **456** quantities recalculated; 48 more prices on a second pass.

5 Labelled what can’t be saved

ERROR / UNKNOWN / blank became “Unknown” (item, payment, location); bad dates became empty.

6 Standardised and checked

9,540 dates converted to YYYY-MM-DD. **0** duplicate IDs. **26** rows flagged for review.

Rows with bad values were kept, not deleted, because their other columns were still good. Only 26 rows (0.26%) stayed unusable.

SQL IN PRACTICE

Rebuilding a missing total from the two columns I could trust

```
UPDATE cafe_sales_clean
SET "Total Spent" = Quantity * "Price Per Unit"
WHERE ("Total Spent" IN ('ERROR', 'UNKNOWN')
       OR "Total Spent" IS NULL)
  AND Quantity NOT IN ('ERROR', 'UNKNOWN')
  AND Quantity IS NOT NULL
  AND "Price Per Unit" NOT IN ('ERROR', 'UNKNOWN')
  AND "Price Per Unit" IS NOT NULL;
```

Only when both inputs are trustworthy

The three AND conditions stop the query from calculating with ERROR, UNKNOWN or empty values.

502 → 23

bad Total Spent values before and after this step

Checked, not assumed

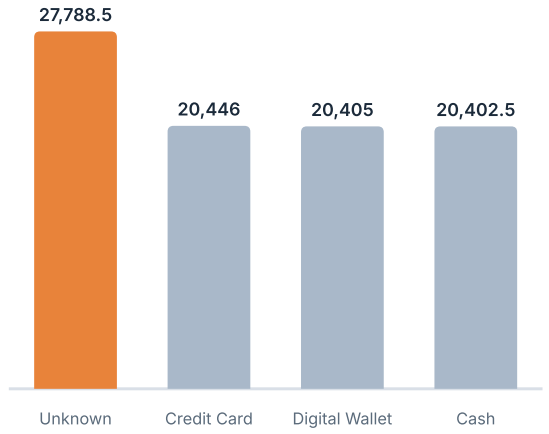
Every UPDATE was compared with the “rows affected” count I expected, and re-audited with a GROUP BY.

THE DASHBOARD

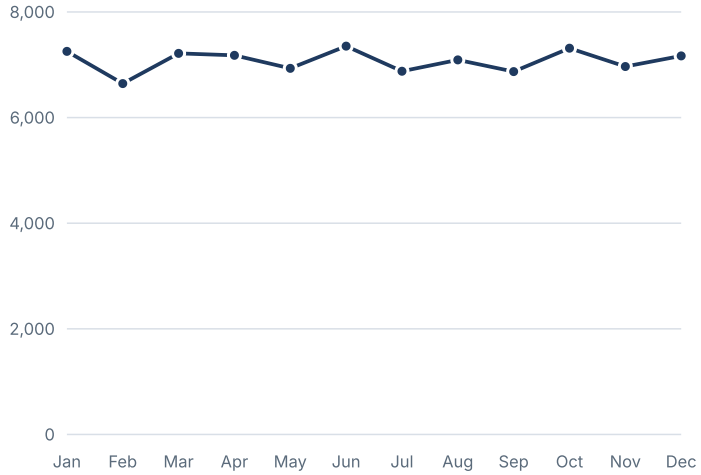
Café Sales 2023, on the 9,974 reliable transactions

TOTAL REVENUE	TRANSACTIONS	AVERAGE SALE
89,042	9,974	8.93

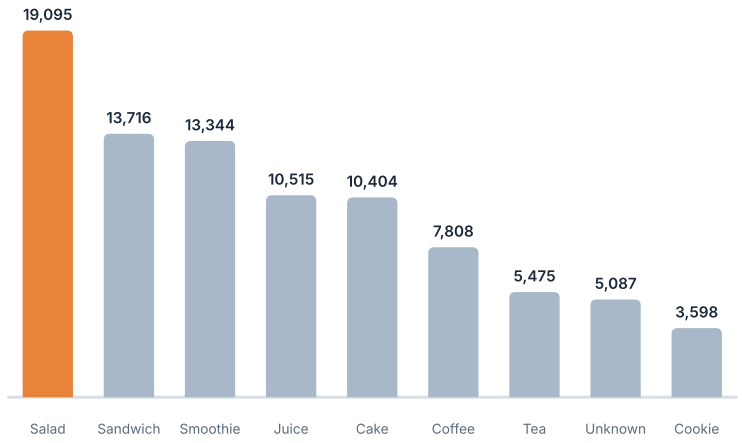
31% of revenue has no payment method recorded



Revenue is steady at about 7,000 a month



Salad earns 21% of revenue



Excludes 26 unrecoverable rows. The monthly chart also leaves out 460 sales with no date. Figures are in the dataset's own currency units.

WHAT THE DATA SAYS

Three findings, and how far to trust each one

21%

Salad leads on revenue, but only because it costs 5

19,095 of 89,042. Each item sold about 1,100–1,300 times, so the ranking follows price, not popularity.

±5%

No seasonality across 2023

Monthly revenue stayed between 6,644 and 7,353, around an average of about 7,070. February looks low only because it has fewer days.

31%

Payment habits are mostly invisible

Cash, credit card and digital wallet each take about 23% of revenue, but 31% of revenue has no payment method at all.

LIMITS AND NEXT STEPS

What this analysis cannot tell us — and what I would do next

Limits

- **31%** of payment methods and **40%** of locations are unknown, so claims about either carry an asterisk.
- About **480** sales have an unknown item: prices 3 and 4 are shared by two items each, so guessing would invent data.
- **460** sales have no date and **26** rows are unusable; both are left out of the relevant charts.
- This is a training dataset with evenly spread values, so the strength of the project is the method, not the business findings.

Next steps

- **Fix it at the source:** require payment method and location at the till, so they stop arriving blank.
- **Add validation** so ERROR and UNKNOWN can never be saved as values.
- **Rebuild the dashboard in Tableau** with filters for item, month and location.
- **Repeat the cleaning in Python** and compare the results with the SQL version.